management schedule and minor inputs

Five small inputs that the larger pages reference but do not document. Each is a single reader populating a single derived type.

management.sch

Fixed-schedule management. Each schedule lists scheduled operations on calendar dates or heat-unit fractions, plus a set of named auto-operation tables (decision tables in `lum.dtl`). `landuse.lum` selects a schedule by name through `mgt`.

- Reader: `src/mgt_read_mgtops.f90` (header) and `src/read_mgtops.f90` (operation rows).
- Default file name: `management.sch` (from `in_lum%management_sch` in `input_file_module.f90`).
- Target type: `management_schedule` in `src/mgt_operations_module.f90`, array `sched(:)`.

File format

- Line 1: title (skipped).
- Line 2: header (skipped).
- For each schedule:
  - One header row: `name`, `num_ops`, `num_autos`.
  - `num_autos` rows, each naming an auto-operation. If the name is `pl_hv_summer1` or `pl_hv_winter1` the row also lists one crop name; if it is `pl_hv_summer2` it lists two crop names.
  - `num_ops` operation rows, list-directed.

Schedule header

#	Field	Type	Description
1	name	char(40)	schedule name referenced by `landuse.lum`
2	num_ops	int	number of scheduled operation rows
3	num_autos	int	number of auto-operation tables

Operation row

Read into `management_ops` (see `mgt_operations_module.f90`).

#	Field	Type	Description
1	op	char(40)	operation code (see below)
2	mon	int	calendar month
3	day	int	calendar day of month
4	husc	real	heat-unit fraction trigger (0 if calendar-driven)
5	op_char	char(40)	name into the operation database keyed by `op`
6	op_plant	char(40)	name into a secondary database (plant, harvest type, chemical application)
7	op3	real	application amount or override value

Recognised `op` codes and the database that `op_char` (and where noted `op_plant`) resolves against:

Code	Action	op_char database	op_plant database
pcom	plant community init	plant.ini (pcomdb)	none
plnt	plant	plants.plt (pldb)	transplant.plt (transpl)
harv	harvest	(none)	harv.ops (harvop_db)
hvkl	harvest and kill	(none)	harv.ops
kill	kill	none	none
till	tillage	tillage.til (tilldb)	none
irrm, irrp	irrigation	irr.ops (irrop_db)	none
fert	fertilizer	fertilizer.frt (fertdb)	chem_app.ops (chemapp_db)
manu	manure	manure_db.frt (manure_db)	chem_app.ops
pest	pesticide	pesticide.pes (pestdb)	chem_app.ops
graz	grazing	graze.ops (grazeop_db)	none
burn	burn	fire.ops (fire_db)	none
swep	street sweep	sweep.ops (sweepop_db)	none
skip	end of rotation year	none	none
prtp	print plant variables	none	none

A skip row advances the internal rotation year counter; subsequent rows belong to the next year of the rotation.

Example

refdata/Ames_sub1/management.sch:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP   MON   DAY HU_SCH   OP_DATA1
 ↪  OP_DATA2  OP_DATA3
mgt_01                                 305         1
                                                   irr_year_irr
                                                       till    4     1    0.0   chisplow
 ↪  null     0.000
                                                       plnt    4    28    0.0       corn
 ↪  null     0
                                                       irrm    5    15    0.0  sprinkler_ilm
 ↪  null     0.000
                                                       fert    6     1    0.0       null
 ↪  elem_n   181.400
                                                       hvkl   10     7    0.0       corn
 ↪  grain     0.000
                                                       skip    0     0    0         null
 ↪  null     0
```

Related files

- landuse.lum selects a schedule by name.
- *.ops hold the operation parameters that op_char and op_plant resolve against.
- *.dtl decision tables provide the auto_name entries.

puddle.ops

Rice paddy puddling parameters. A puddling event resets the upper soil-layer hydraulic conductivity and the in-paddy water chemistry to fixed values.

- Reader: src/mgt_read_puddle.f90.
- File name: hard-coded puddle.ops.
- Target type: puddle_operation in src/mgt_operations_module.f90, array pudl_db(:).

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record, list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	wet_hc	real	0	mm/h	upper-layer hydraulic conductivity after puddling
3	sed	real	0	ppm	sediment concentration after puddling
4	orgn	real	0	ppm	organic N concentration after puddling
5	sedp	real	0	ppm	organic P concentration after puddling
6	no3	real	0	ppm	NO3-N concentration after puddling
7	solp	real	0	ppm	mineral (soluble) P concentration after puddling
8	nh3	real	0	ppm	NH3 concentration after puddling
9	no2	real	0	ppm	NO2 concentration after puddling

Related files

- `*.ops` for the other operation databases that follow the same skip-two-and-read pattern.

transplant.plt

Plant transplant database. Initial state for a plant that is established by transplant rather than from seed. Operation rows with `op = plnt` look up `op_plant` against this database.

- Reader: `src/plant_transplant_read.f90`.
- File name: hard-coded `transplant.plt`.
- Target type: `plant_transplant_db` in `src/plant_data_module.f90`, array `transpl(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record.

#	Field	Type	Default	Units	Description
1	name	char(40)	`frsd`	none	transplant entry name
2	lai	real	0	$m^{2/m}2$	initial leaf area index
3	bioms	real	0	kg/ha	initial biomass
4	phuacc	real	0	fraction	initial fraction of accumulated heat units
5	fr_yrmat	real	0.05	fraction	fraction of current year of growth to years to maturity
6	pop	real	0	plants/m^2	initial plant population

Related files

- plants.plt for the underlying plant parameter database.
- management.sch resolves the op_plant field against transplant.plt for plnt operations.

co2.out and co2_yr.dat

CO2 driver. Despite the .out extension, co2_read is the routine that establishes the per-year atmospheric CO2 series used by the simulation. The reader reads co2_yr.dat if present, fills the per-year array co2y(:), and writes the resolved series to co2.out as a record of what was used.

- Reader: src/co2_read.f90.
- Input file (optional): co2_yr.dat.
- Output file: co2.out (unit 2222).
- Fallback: when co2_yr.dat is missing, every simulation year is filled with bsn_prm%co2 from parameters.bsn.

co2_yr.dat format

Line	Content
1	title (skipped)
2	integer yrs, the number of year records that follow
3	header (skipped)
4 to 3+yrs	one record per year: iyr (int, calendar year) and co2 (real, ppm)

How the series is built

For a simulation that runs time%nbyr years starting in time%yrc_start:

- If co2_yr.dat is absent or empty, every simulation year is set to bsn_prm%co2.
- If the latest year in co2_yr.dat is at or before the simulation start, every simulation year is set to that final value.
- Otherwise the file is aligned to the simulation start. Simulation years that fall before the first record use the first record's value; years that fall after the last record carry the last value forward.

co2.out format

Header YR CO2(ppm), then one row per simulation year with the calendar year and the resolved ppm value.

Related files

- codes.bsn and parameters.bsn for the co2 fallback in bsn_prm.

object.prt

Per-object output selector. Each row asks the model to write one specific hydrograph or state variable for one spatial object to a named file. Reuses the old saveconc mechanism.

- Reader: src/object_read_output.f90.
- Default file name: object.prt (from in_sim%object_prt in input_file_module.f90).
- Target type: object_output in src/hydrograph_module.f90, array ob_out(:).

File format

Line 1 title and line 2 header are skipped. Each remaining row is one selection.

#	Field	Type	Description
1	k	int	row id (the reader also uses it to size the array)
2	obtyp	char(10)	object type (see below)

#	Field	Type	Description
3	obtypno	int	object number within `obtyp` (1-based)
4	hydtyp	char(20)	hydrograph or state code (see below)
5	filename	char(26)	output file name to open and write to

Object types recognised by the reader:

Code	Object
`hru`	HRU
`hlt`	HRU LTE
`ru`	routing unit
`res`	reservoir
`cha`	channel
`exc`	export coefficient object
`dr`	delivery ratio object
`out`	outlet
`sdc`	SWAT-Deg channel

Hydrograph or state codes:

Code	Content	Header written
`tot`	total flow	`hyd_hdr_time hyd_hdr`
`rhg`	recharge	`hyd_hdr_time hyd_hdr`
`sur`	surface	`hyd_hdr_time hyd_hdr`
`lat`	lateral	`hyd_hdr_time hyd_hdr`
`til`	tile	`hyd_hdr_time hyd_hdr`
`sol_water`	soil moisture by layer	`hyd_hdr_time sol_hdr`
`solnut_ly`	soil N and P by layer	(no header)
`solnut_pr`	soil N and P for profile	(no header)
`plant`	plant status	`hyd_hdr_time plt_hdr plt_hdr`
`cha_fp`	channel and flood-plain water balance	`hyd_hdr_time fp_hdr`

Each row opens its own file at unit `6100 + i` and writes the matching header. The simulation appends to that file as the run progresses.

Related files

- `print.prt` controls the main output selection. `object.prt` is the side channel for per-object hydrograph files.